Coder Army

AI Application Testing : Deterministic Evals & LLM-as-a-Judge

Traditional software is mostly deterministic.

If we give the same input to a function, we usually expect the same output.

AI applications are different.

An AI system may involve:

- LLM reasoning
- RAG
- Tool calling
- External data
- Non-deterministic model responses

So testing only the final output is often not enough.

We also want to know:

- Did the correct document get retrieved?
- Did the model call the correct tool?
- Were the correct arguments passed?
- Was the answer grounded in the available evidence?
- Did the model hallucinate?
- Was the answer actually useful?

This is where **AI Evals** become important.

---

1. Application We Will Evaluate

Coder Army

We will build a small:

E-commerce Support Agent

The agent answers three kinds of questions.

---

2. Category A — Knowledge-Based Questions

Example:

```
How many days do I have to return a product?
```

The answer exists inside company documents.

For example:

```
Return Policy

Products can be returned within
30 days of delivery.
```

The LLM should not be expected to know our company's private policies.

Instead, the application retrieves the information using RAG.

```
Question
   ↓
Knowledge Base
   ↓
RAG
   ↓
Answer
```

This creates two separate things we can evaluate.

Retrieval Evaluation

Did the application retrieve the correct document?

```
Expected document:
return-policy
```

Generation Evaluation

After getting the correct document, did the model answer correctly?

These are two different failure points.

A wrong answer could mean:

```
Wrong document retrieved
```

or:

```
Correct document retrieved
but the LLM generated a bad answer
```

This distinction is extremely useful while debugging AI applications.

---

3. Category B — Live / Order-Specific Questions

Consider:

```
Where is ORD-101?
```

This information cannot realistically come from our policy documents.

A policy document might say:

```
Orders usually arrive within 3–5 business days.
```

Coder Army

But it cannot tell us:

```
ORD-101 is currently shipped.
```

That is dynamic, application-specific information.

For this, we need a tool.

```
Question
   ↓
LLM
   ↓
getOrderStatus("ORD-101")
   ↓
Tool Result
   ↓
LLM
   ↓
Answer
```

Now we get two more behaviours that can be evaluated.

Tool Selection

```
Did the model call getOrderStatus?
```

Tool Arguments

```
Did the model pass ORD-101?
```

Both are deterministic checks.

---

4. Category C — Questions That Need Both RAG and Tools

Consider:

Coder Army

Coder Army

```
My order ORD-102 was delivered 20 days ago.
Can I still return it?
```

The application needs two different pieces of information.

Information 1 — Company Policy

The return policy may come from RAG.

```
Products may be returned within
30 days of delivery.
```

Information 2 — Order Information

The actual order information may come from a tool.

```
ORD-102
Status: Delivered
Delivered: 20 days ago
```

The model then combines both pieces of information:

```
20 days
<
30-day return window
```

and answers:

```
Yes. Your order is still within
the 30-day return window.
```

Now the application genuinely involves:

```
RAG
+
Tools
```

```
+
LLM Reasoning
```

This makes it a good application for understanding AI evaluations.

5. Why Not Just Test a Simple Chatbot?

Suppose our application was only:

```
User
 ↓
LLM
 ↓
Answer
```

We could evaluate things such as:

- Relevance
- Correctness
- Helpfulness

But there would not be much internal behaviour to inspect.

Our support agent gives us many observable behaviours:

```
Did retrieval happen correctly?

Did the correct document come back?

Did the correct tool get called?

Were the tool arguments correct?

Was the final answer relevant?

Was the answer correct?

Was it grounded?

Did the model hallucinate?
```

Coder Army

This gives us two major categories of evaluation:

```
Deterministic Evals
+
LLM-as-a-Judge
```

---

6. Tools Used by the Support Agent

We can keep the tools very small.

Tool 1 — Order Status

```
getOrderStatus(orderId)
```

Example:

```
getOrderStatus("ORD-101")
```

Result:

```
SHIPPED
```

Tool 2 — Delivery Estimate

```
getDeliveryEstimate(orderId)
```

Example:

```
getDeliveryEstimate("ORD-101")
```

Coder Army

Result:

```
Expected delivery: 15 October 2026
```

Tool 3 — Refund Amount

```
getRefundAmount(orderId)
```

Example:

```
getRefundAmount("ORD-103")
```

Result:

```
₹4,999
```

---

7. Basic Application Architecture

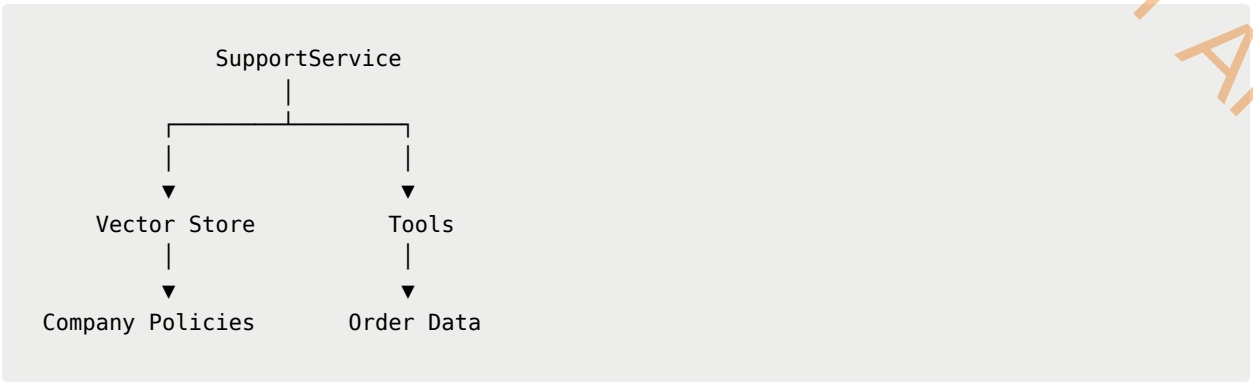
At the application level:

```
USER
 |
 ▼
POST /chat
 |
 ▼
SupportController
 |
 ▼
SupportService
 |
 ▼
ChatClient
```

Coder Army

Coder Army

The `SupportService` can access two major external information sources:



The LLM can use either source depending on the question.

---

8. Observable Agent Behaviour

Suppose the API only returns:

```
Yes, you can return it.
```

The user may be satisfied, but from a testing perspective we know almost nothing.

We do not know:

- Which document was retrieved
- Whether retrieval happened at all
- Whether a tool was called
- Which tool was called
- Which arguments were passed
- What context was given to the LLM
- Whether the model simply guessed

This becomes a serious testing problem.

An AI application is not just:

Coder Army

```
Input → Output
```

It is usually a pipeline.

```
Question
 ↓
Retrieval
 ↓
Tool Decision
 ↓
Tool Result
 ↓
Generation
 ↓
Answer
```

If we only test the final answer, we cannot easily determine where the failure occurred.

---

9. AgentResponse and Eval Trace

For evaluations, the application should expose a small amount of internal execution information.

Conceptually, we can return an:

```
AgentResponse
```

containing information such as:

```
answer

toolsUsed

toolArguments

retrievedDocuments
```

Coder Army

```
context
```

Example:

```
answer:
"Products may be returned within 30 days of delivery."

toolsUsed:
[]

retrievedDocuments:
["return-policy"]
```

Another example:

```
answer:
"ORD-101 has been shipped."

toolsUsed:
["getOrderStatus"]

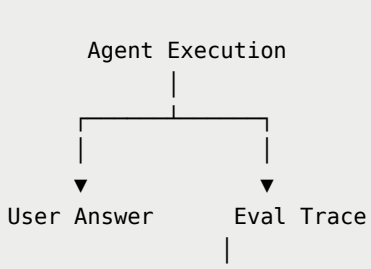
retrievedDocuments:
[]
```

The user does not necessarily need to see this trace.

It is mainly useful for:

- Testing
- Debugging
- Observability

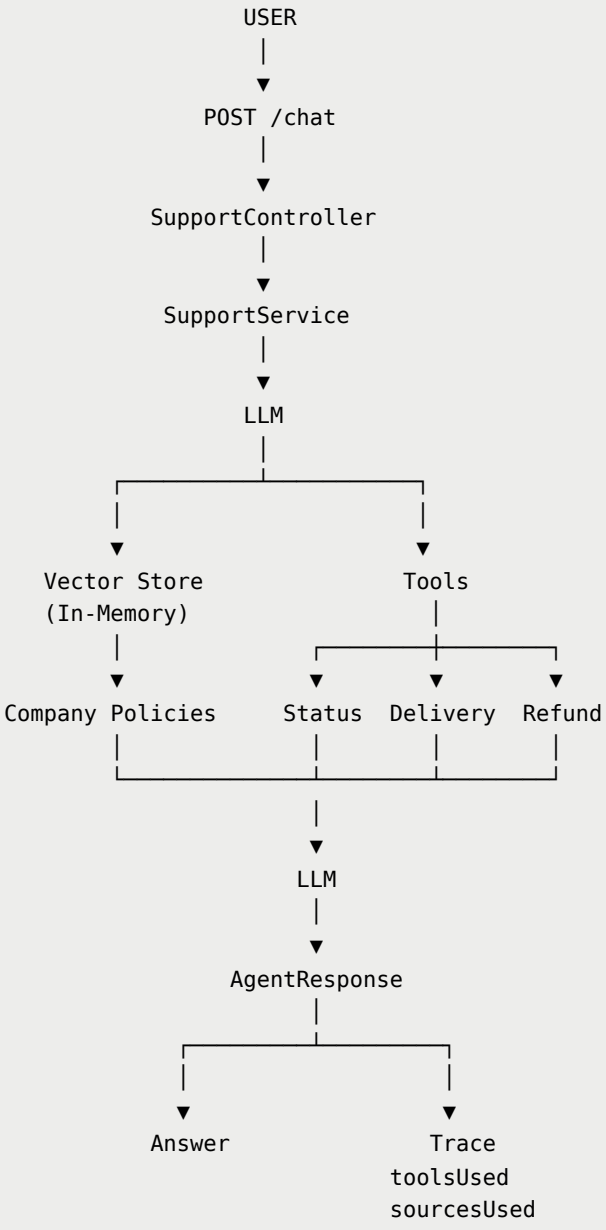
So the system effectively produces two outputs.



Coder Army

```
                    ├── toolsUsed
                    ├── toolArguments
                    ├── sourcesUsed
                    └── context
```

10. Final Application Architecture



Coder Army

11. Why Store `retrievedDocuments`?

Suppose the golden dataset says:

```
expectedSource:
return-policy
```

And the actual response contains:

```
retrievedDocuments:
["return-policy"]
```

The evaluation becomes simple:

```
Expected:
return-policy

Actual:
return-policy

PASS
```

No LLM judge is required.

This is deterministic behaviour.

---

12. Why Store `context`?

The context may not be useful to the end user, but it becomes extremely useful during evaluation.

For example, a **faithfulness judge** needs:

```
Question

Retrieved Context

Generated Answer
```

Coder Army

So later we can directly access:

```
AgentResponse response =
        supportService.chat(question);

response.getContext();
response.getAnswer();
```

and provide both to the judge model.

The judge can then check whether the answer is supported by the evidence that was actually available to the model.

---

13. Why Store `toolsUsed`?

Suppose the golden dataset says:

```
expectedTool:
getOrderStatus
```

And the actual trace says:

```
toolsUsed:
getOrderStatus
```

Then:

```
PASS
```

Again, there is no reason to ask another LLM whether the correct tool was used.

We already know exactly what happened.

---

14. Why Store `toolArguments`?

Correct tool selection and correct tool arguments are two different behaviours.

Coder Army

Consider:

```
getOrderStatus("ORD-999")
```

for the question:

```
Where is ORD-101?
```

The model selected the correct tool.

```
Tool Selection:
PASS
```

But it passed the wrong argument.

```
Tool Argument:
FAIL
```

Without storing tool arguments, this distinction would be lost.

---

15. Data Available for Evaluation

Our application now gives us three layers of information.

Input

```
question
```

System Execution

```
retrievedDocuments
toolsUsed
```

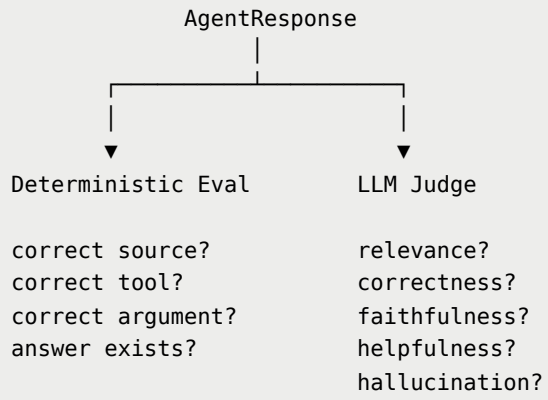
Coder Army

```
toolArguments
context
```

Output

```
answer
```

This gives us enough information for two types of evaluation.



---

16. Golden Dataset

The next step is to create a set of known test cases.

For example:

```
src/test/resources/golden-dataset.json
```

A golden dataset contains examples for which we already know the expected behaviour.

A test case may contain information such as:

```
{
  "question": "Where is ORD-101?",
  "expectedTool": "getOrderStatus",
```

```
  "expectedToolArgument": "ORD-101",
  "expectedSource": null,
  "goldenAnswer": "The order has been shipped."
}
```

The exact structure depends on the application.

The important idea is that the dataset defines what we consider correct behaviour.

---

17. Running the Real Application During Evaluation

For every test case, we run the actual application.

```
AgentResponse response =
        supportService.chat(
                evalCase.getQuestion()
        );
```

The goal is not to mock the AI layer.

We want the actual system to execute:

```
LLM
RAG
Tools
```

because those are the parts we are trying to evaluate.

The result may contain:

```
answer
retrievedDocuments
toolsUsed
toolArguments
context
```

Now we can inspect both the execution and the final response.

Coder Army

Coder Army

18. Deterministic Evaluations

Some properties of an AI application are completely objective.

For example:

```
Did the expected document get retrieved?
```

There is no semantic judgement involved.

Similarly:

```
Did getOrderStatus get called?
```

and:

```
Was ORD-101 passed to the tool?
```

These have exact answers.

The main principle is:

> **Anything we can reliably evaluate using normal code should be evaluated using normal code.**

There is no need to spend another LLM call on something we already know exactly.

---

19. Deterministic Checks in Our Demo

We will evaluate:

```
Response exists

Correct tool was used

Correct tool argument was used
```

Coder Army

```
Expected RAG document was retrieved
```

These checks behave like normal software tests.

Example:

```
Response exists           PASS

Correct tool              PASS

Correct tool argument     PASS

Expected source           PASS
```

---

20. What Should Not Be Tested Deterministically?

Consider questions such as:

```
Is the answer correct?

Is the answer useful?

Is it grounded in the evidence?

Did it properly answer the user's question?
```

These require understanding natural language and meaning.

A simple:

```
expected.equals(actual)
```

will not work well.

Two answers can mean the same thing while using completely different words.

For these situations, we can use:

Coder Army

21. LLM-as-a-Judge

The idea is simple.

Our main AI application generates an answer.

Then another model evaluates that answer according to clearly defined criteria.

The judge may receive:

```
Question

Golden Answer

Expected Behaviour

RAG Context

Tool Results

Actual Answer
```

Conceptually:

```
Golden Dataset
      |
      ▼
Question
      |
      ▼
AI Application
      |
      ▼
Generated Answer
      |
      ├──────── RAG Evidence
      |
      └──────── Tool Evidence
      |
      ▼
   Judge LLM
      |
```

Coder Army

```
      ▼
Quality Scores
```

22. Quality Metrics

For this application, we can evaluate five dimensions.

Metric	What are we checking?
Relevance	Did the response actually answer what the user asked?
Correctness	Is the answer correct compared with the expected behaviour?
Faithfulness	Are the factual claims supported by RAG or tool evidence?
Helpfulness	Is the response clear and useful to the user?
Hallucination-Free	Did the model avoid inventing unsupported information?

23. Use `hallucinationFree` Instead of `hallucination`

Instead of defining:

```
hallucination
```

it is easier to define:

```
hallucinationFree
```

because now all metrics follow the same scoring direction.

```
5 = Good
1 = Bad
```

For example:

Coder Army

```
Relevance           5 = good
Correctness         5 = good
Faithfulness        5 = good
Helpfulness         5 = good
HallucinationFree   5 = good
```

This makes evaluation and aggregation simpler.

---

24. Structured Output for the Judge

We do not want the judge to return something vague like:

```
"The answer seems pretty good overall..."
```

That is difficult for our Java application to process.

Instead, the judge should return structured output.

Example:

```
{
  "relevance": 5,
  "correctness": 5,
  "faithfulness": 4,
  "helpfulness": 5,
  "hallucinationFree": 5,
  "reason": "The answer correctly uses the supplied return policy and does not introduce
unsupported information."
}
```

Now the application can:

- Read individual scores
- Apply thresholds
- Calculate averages
- Compare different prompts or models
- Store evaluation results

- Generate reports

Coder Army

25. The Judge Prompt Matters

The quality of LLM-as-a-Judge depends heavily on the judge prompt.

A vague instruction such as:

```
Tell me if this answer is good.
```

will produce vague and inconsistent results.

The judge should clearly know:

- What information it can use
- What each metric means
- What the scoring scale means
- Whether it should compare with a golden answer
- How it should treat unsupported claims
- What output format is expected

The judge itself is still an LLM.

So the evaluation instructions need to be carefully designed.

26. Complete Evaluation Pipeline

Instead of thinking of the evaluation code as one large test class, it is easier to understand it as a pipeline.

Step 1 — Load the Golden Dataset

```
Golden Dataset
```

Coder Army

This gives us a collection of known test cases.

For example:

```
8 known behaviours
```

Step 2 — Pick a Test Case

Example:

```
How many days do I have to return a product?
```

Step 3 — Run the Real Application

```
supportService.chat(question);
```

The real:

```
LLM
RAG
Tools
```

are allowed to execute.

Step 4 — Inspect the Execution

Suppose the result is:

```
retrievedDocuments:
return-policy

toolsUsed:
[]
```

```
answer:
Products may be returned within 30 days.
```

We now have enough information to evaluate the execution.

27. Layer 1 — Deterministic Evaluation

Example:

```
Response exists             PASS

No unexpected tool          PASS

Expected source retrieved   PASS
```

These are normal software tests.

No second LLM is required.

28. Layer 2 — Quality Evaluation

Next, send the relevant information to the judge.

```
Question

Golden Answer

Expected Behaviour

RAG Evidence

Tool Evidence

Generated Answer
```

The judge may return:

```
Relevance           5
Correctness         5
```

Coder Army

```
Faithfulness        5
Helpfulness         4
Hallucination-Free  5
```

These metrics evaluate the semantic quality of the response.

29. Layer 3 — Quality Threshold

For this demo, we can define:

```
Score >= 4
```

as acceptable for every quality metric.

There is nothing universal about the number `4`.

It is simply a product requirement for this application.

Different systems may require different thresholds.

For example:

Healthcare or Financial Applications

We may need much stricter requirements because incorrect information can have serious consequences.

Creative Applications

Faithfulness may be less important because creativity is part of the expected behaviour.

So thresholds are:

> **Product and business decisions, not fundamental constants.**

30. Layer 4 — Overall Pass / Fail

The final test combines both layers.

Coder Army

Coder Army

```
Deterministic Checks
       PASS

       AND

Quality Scores >= 4
       PASS

        ↓

     TEST PASS
```

If either layer fails:

```
TEST FAIL
```

This is important because a response can fail in different ways.

For example:

```
Correct final answer
but wrong tool used
```

or:

```
Correct source retrieved
but final answer incorrect
```

or:

```
Correct tool and source
but hallucinated extra information
```

A good evaluation system helps us identify exactly what went wrong.

---

31. Deterministic Evals vs LLM-as-a-Judge

Coder Army

A useful rule is:

Use deterministic evaluation when the answer is objectively knowable.

Examples:

```
Was a response returned?

Was getOrderStatus called?

Was ORD-101 passed?

Was return-policy retrieved?

Was the JSON schema valid?
```

Use an LLM judge when understanding meaning is required.

Examples:

```
Was the answer relevant?

Was it correct?

Was it faithful to the evidence?

Was it helpful?

Did it hallucinate?
```

Do not use an LLM judge for something that can be checked reliably with normal code.

---

32. Why AI Evals Are Different From Traditional Testing

Traditional testing often looks like:

Coder Army

```
Input
 ↓
Function
 ↓
Expected Output
```

AI systems look more like:

```
Input
 ↓
Retrieval
 ↓
Tool Selection
 ↓
Tool Arguments
 ↓
External Data
 ↓
LLM Reasoning
 ↓
Generated Answer
```

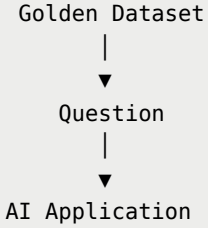
So a useful AI evaluation strategy tests both:

```
Behaviour of the system
+
Quality of the final response
```

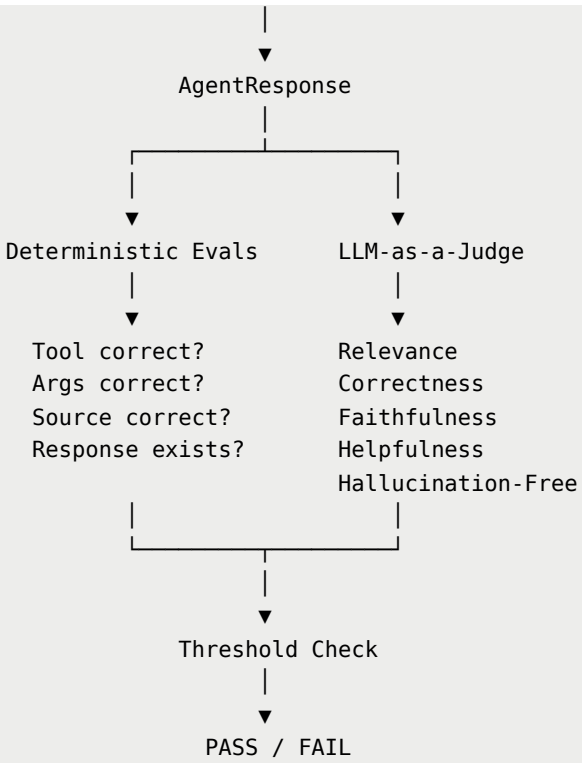
This is why **observability** becomes a key part of building production AI applications.

---

33. Final Evaluation Architecture



Coder Army



34. Key Takeaways

1. Do not test only the final answer

AI applications are pipelines. A wrong answer may originate from retrieval, tool selection, tool arguments, context, or generation.

2. Make important agent behaviour observable

Keep track of information such as:

```
retrievedDocuments
toolsUsed
toolArguments
context
answer
```

3. Use normal code wherever possible

If something can be checked exactly, use deterministic evaluation.

```
Expected Tool == Actual Tool
```

is better than asking another LLM whether the correct tool was used.

4. Use LLM-as-a-Judge for semantic quality

LLM judges are useful for evaluating:

```
Relevance
Correctness
Faithfulness
Helpfulness
Hallucination-Free
```

5. Use structured judge output

Structured output makes scores easy to process, compare, aggregate, and store.

6. Build a golden dataset

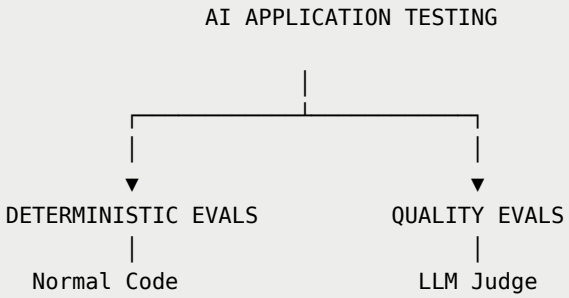
A useful evaluation system starts with known questions and expected behaviours.

7. Define thresholds according to the product

There is no universal passing score.

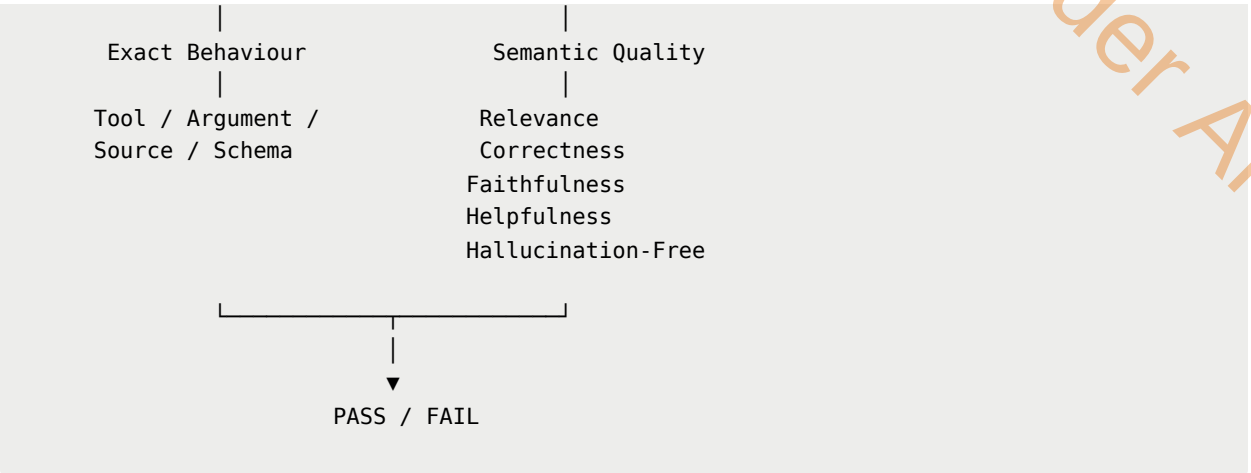
The acceptable quality level depends on the application and its risk.

Final Mental Model



Coder Army

Coder Army



The core idea is simple:

> **Test deterministic behaviour with deterministic code. Use LLM-based evaluation only where understanding language and meaning is actually required.**